FACULDADE DE TECNOLOGIA DE COTIA

ENZO FERNANDES
VICTOR RAFAEL DE OLIVEIRA DIAS

DIGITAL TWIN INDUSTRIAL
BASE DE DADOS: DER, MODELAGEM E CRUD

Cotia - SP
2026

Sumário

1. Resumo da entrega 3
2. Banco escolhido: PostgreSQL 4
3. O que o sistema precisa guardar 6
4. Entidades e relacionamentos 7
5. Diagramas entidade-relacionamento (DER) 8
5.1 Visão geral 9
5.2 Módulo Acesso 10
5.3 Módulo Catálogo de máquinas 11
5.4 Módulo Apoio ao usuário 12
5.5 Módulo Fábricas 12
5.6 Módulo Simulação e resultados 13
6. Normalização e decisões de projeto 14
7. CRUD: Create, Read, Update e Delete 15
8. Dicionário de dados 16
Módulo: Acesso 16
Módulo: Catálogo de máquinas 16
Módulo: Apoio ao usuário 21
Módulo: Fábricas 22
Módulo: Simulação e resultados 24
9. Como executar 30

1. Resumo da entrega

O Sprint 1 pede a **base de dados** do sistema: o diagrama entidade-relacionamento (DER), a própria base com as tabelas criadas e a capacidade de executar comandos de CRUD (Create, Read, Update e Delete). Tudo está no repositório do projeto, na pasta `banco-de-dados/`.

Pedido	Onde está
Diagrama entidade-relacionamento (DER)	Pasta `der/`: visão geral, um diagrama por módulo e o modelo completo (neste documento, na seção 5)
A base de dados com as tabelas criadas	`digital_twin.db` (SQLite, cópia portátil) e os scripts em `sql/` (SQLite e PostgreSQL). 21 tabelas, 4 views e 345 linhas iniciais
Comandos de CRUD	`sql/04_crud_exemplos.sql` e `scripts/crud_demo.py`, que executa Create, Read, Update e Delete em todas as tabelas
Testes	`testes/test_banco.py`: 23 testes automáticos no SQLite (criação, dados, CRUD e regras de integridade) e `scripts/testar_postgres.py`: a mesma verificação num servidor PostgreSQL 16 (Docker), que passou
Documentação	Este documento, `docs/modelagem.md` e `docs/dicionario-de-dados.md`

O banco foi pensado para o sistema Digital Twin Industrial, um simulador de fábrica de bebidas no estilo Cisco Packet Tracer. Ele guarda o catálogo de máquinas (com limites, capacidade, consumo, preços estimados e fontes), as fábricas montadas pelos usuários, os resultados de simulação, os planos de melhoria, o registro de eventos e a conversa com o assistente.

2. Banco escolhido: PostgreSQL

A escolha do banco foi livre. **O banco do sistema é o PostgreSQL.** O SQLite entra só como cópia portátil para demonstração (abre com duplo clique, sem instalar servidor).

Critério	PostgreSQL	SQLite	MySQL
Vários usuários e a API gravando ao mesmo tempo	Sim (servidor, transações e bloqueios por linha)	Limitado (um escritor por vez)	Sim
Regras de integridade que o modelo usa (chave estrangeira composta, CHECK entre colunas, ON DELETE)	Completo	Completo, mas as chaves estrangeiras precisam ser ligadas a cada conexão	CHECK e chaves compostas com limitações históricas
Combina com o backend (Python, FastAPI, SQLAlchemy)	Excelente	Bom (só para protótipo)	Bom
Tipos para o que o sistema guarda (NUMERIC exato, TIMESTAMP, BOOLEAN, JSON)	Completo, com JSONB	Tipos fracos (tudo vira poucos tipos internos)	Bom
Busca por significado nos manuais (assistente RAG, previsto no projeto)	Extensão `pgvector` no próprio banco	Não tem	Recurso mais recente
Custo e licença	Gratuito e código aberto	Gratuito e de domínio público	Gratuito (Oracle)
Facilidade para o professor abrir	Precisa de servidor (Docker ou instalação)	Basta o arquivo	Precisa de servidor

Por que PostgreSQL para esta aplicação:

1. **O sistema vai ter uma API gravando e lendo ao mesmo tempo**, com vários usuários e simulações. O SQLite trava com vários escritores; o PostgreSQL foi feito para isso.
2. **A integridade importa**: o modelo usa chaves compostas (uma conexão só liga máquinas da mesma fábrica), regras entre colunas e apagamento em cascata. O PostgreSQL aplica tudo isso sempre, sem configuração extra.
3. **O projeto já previa o PostgreSQL** (decisão `docs/decisoes/0001-stack.md`), então não há troca de rumo.
4. **O assistente com leitura de manuais (RAG)**, que está no escopo, pode usar a extensão `pgvector` para buscar trechos por significado, sem outro banco.
5. É **gratuito, aberto e padrão de mercado**, o que ajuda no futuro (hospedagem, equipe, documentação).

Como ficou a entrega:

- Os scripts de criação, dados, views e exemplos de CRUD existem para **os dois**, gerados do mesmo modelo (`scripts/modelo.py`), então nunca saem diferentes. Os dados, as views e o CRUD usam SQL padrão e rodam igual nos dois bancos.
- O `digital_twin.db` (SQLite) é a cópia portátil: serve para o professor abrir, para a demonstração de CRUD e para os testes automáticos.

- **Os scripts de PostgreSQL foram executados num servidor PostgreSQL 16 (Docker) e passaram:** as 21 tabelas, as 4 views e os 345 registros iniciais foram criados sem erro; o CRUD completo (`sql/04_crud_exemplos.sql`) rodou e deixou o banco igual ao inicial; e o banco recusou os 14 comandos inválidos de `testes/test_integridade_postgres.sql` (chaves estrangeiras, valores únicos, regras CHECK, chave composta e RESTRICT). O `docker-compose.yml` sobe o banco com um comando (`docker compose up -d`) e `scripts/testar_postgres.py` repete essa verificação.
- A validação foi feita com o PostgreSQL 16.15 em contêiner Linux (Alpine). Os testes unitários em Python (`testes/test_banco.py`) continuam rodando no SQLite.

3. O que o sistema precisa guardar

A partir das telas do front-end, o sistema precisa lembrar de:

Necessidade	Vem de qual tela ou função	Tabelas
Quem entra no sistema	Login	`usuarios`
As máquinas que existem para montar a fábrica, com limites, capacidade e consumo	Lista de máquinas, aba Máquina, Configurações › Limites	`categorias`, `etapas`, `maquinas_catalogo`
Explicação de cada máquina	Manual das máquinas	`manuais_maquina`
De onde vêm os números (com link e confiança)	"De onde vêm os números", Catálogo e fontes	`fontes`, `maquina_fontes`
Preço estimado de uma melhoria	Resultados › Avisos, Configurações › Preços	`precos_maquina`
Modelos comerciais reais pesquisados	Pesquisa de mercado do projeto	`modelos_reais`
Palavras difíceis explicadas	Glossário e termos sublinhados	`glossario_termos`, `glossario_palavras`
A fábrica montada pelo usuário	Aba Montar	`fabricas`, `fabrica_maquinas`, `conexoes`
Fotos "antes e depois" de uma fábrica	Resultados › Antes × depois	`cenarios`, `cenario_maquinas`
Resultado de cada Play	Janela de resultado final	`simulacoes`
O que aconteceu e quando	Aba Registro	`eventos_registro`
Planos do tipo "quero +10%"	Aba Objetivo, simular uma melhoria	`planos_objetivo`, `plano_passos`
Conversa com o assistente	Aba Conversa	`mensagens_chat`

4. Entidades e relacionamentos

Cada linha lê-se "um(a) X … vários(as) Y".

Relacionamento	Cardinalidade	Observação
`usuarios` possui `fabricas`	1 para N	apagar o usuário apaga as fábricas dele (se não houver histórico de simulação)
`categorias` agrupa `maquinas_catalogo`	1 para N	
`etapas` contém `maquinas_catalogo`	1 para N	a etapa também diz em que zona da planta baixa a máquina aparece
`maquinas_catalogo` tem `manuais_maquina`	1 para 1	a chave primária do manual é a própria chave estrangeira
`maquinas_catalogo` N para N `fontes`	N para N	tabela de ligação `maquina_fontes`, com a finalidade (`numeros` ou `preco`)
`maquinas_catalogo` tem `precos_maquina`	1 para N	permite guardar o histórico de preços
`maquinas_catalogo` é exemplificada por `modelos_reais`	1 para N	cada modelo real aponta para uma fonte
`glossario_termos` é chamado por `glossario_palavras`	1 para N	
`fabricas` contém `fabrica_maquinas`	1 para N	cada uma é "um tipo do catálogo colocado na fábrica"
`maquinas_catalogo` instancia `fabrica_maquinas`	1 para N	
`fabrica_maquinas` liga-se a `fabrica_maquinas` por `conexoes`	N para N (auto-relacionamento)	origem e destino da mesma fábrica
`fabricas` gera `cenarios`; `cenarios` detalha `cenario_maquinas`	1 para N, 1 para N	
`fabricas` é simulada em `simulacoes`; recebe `planos_objetivo`	1 para N	
`planos_objetivo` é feito de `plano_passos`	1 para N	
`simulacoes` e `fabricas` registram `eventos_registro`	1 para N (opcional)	o evento pode ser geral, de uma fábrica ou de uma simulação
`usuarios` escreve `mensagens_chat`	1 para N	

O banco tem 21 tabelas, organizadas em 5 módulos:

Módulo	Tabelas
Acesso	`usuarios`
Catálogo de máquinas	`categorias`, `etapas`, `maquinas_catalogo`, `manuais_maquina`, `fontes`, `maquina_fontes`, `precos_maquina`, `modelos_reais`
Apoio ao usuário	`glossario_termos`, `glossario_palavras`, `mensagens_chat`
Fábricas	`fabricas`, `fabrica_maquinas`, `conexoes`
Simulação e resultados	`cenarios`, `cenario_maquinas`, `simulacoes`, `planos_objetivo`, `plano_passos`, `eventos_registro`

5. Diagramas entidade-relacionamento (DER)

Os diagramas usam a notação "pé de galinha": `||` significa exatamente um, `|o` zero ou um e `o{` zero ou muitos. PK é a chave primária e FK é a chave estrangeira. A visão geral mostra as 21 tabelas com as chaves; os diagramas por módulo mostram todas as colunas e são os mais fáceis de ler. As imagens também estão em alta resolução na pasta `der/` do repositório.

5.1 Visão geral

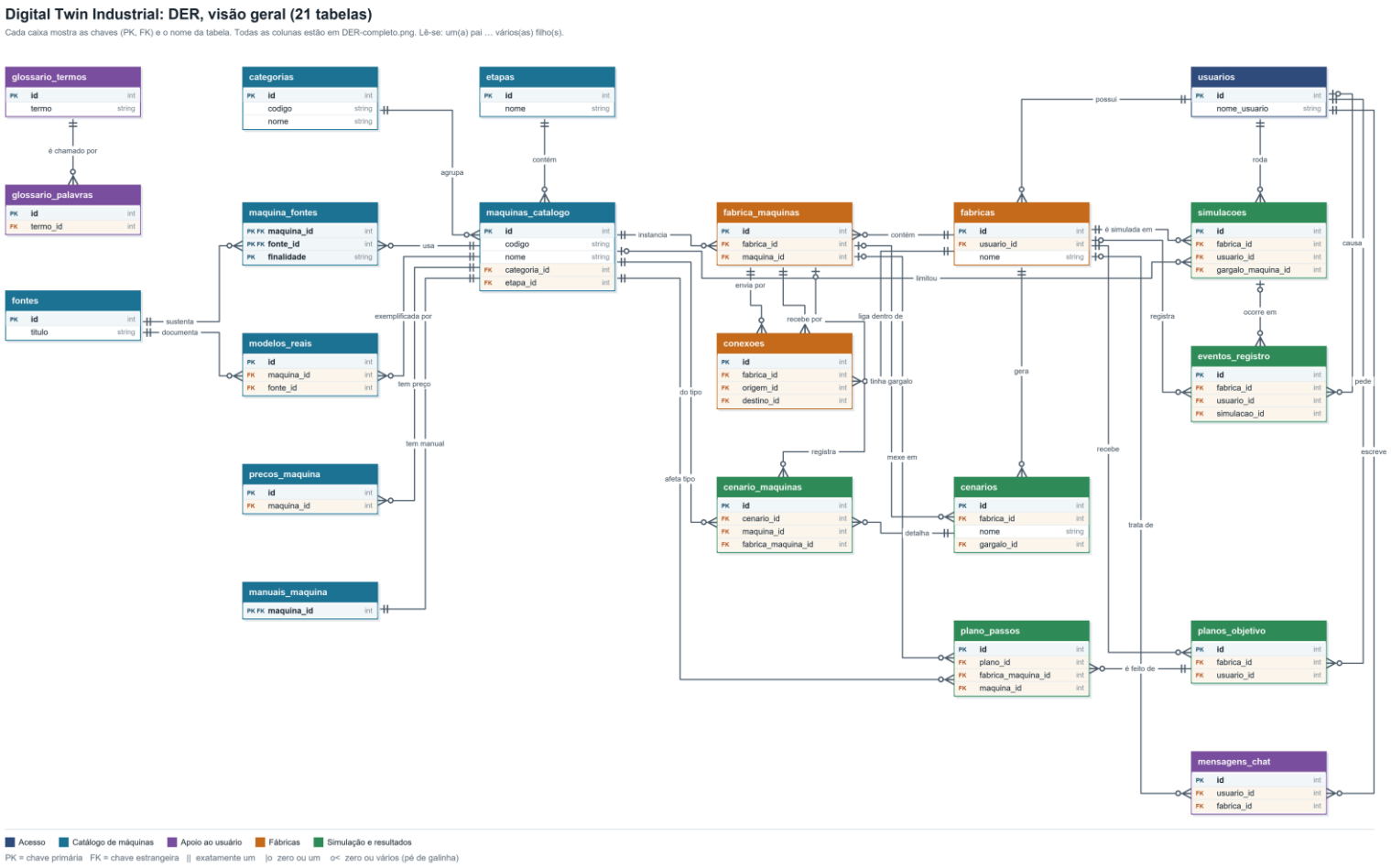


Figura 1 – DER: visão geral das 21 tabelas, com as chaves primárias e estrangeiras.

5.2 Módulo Acesso

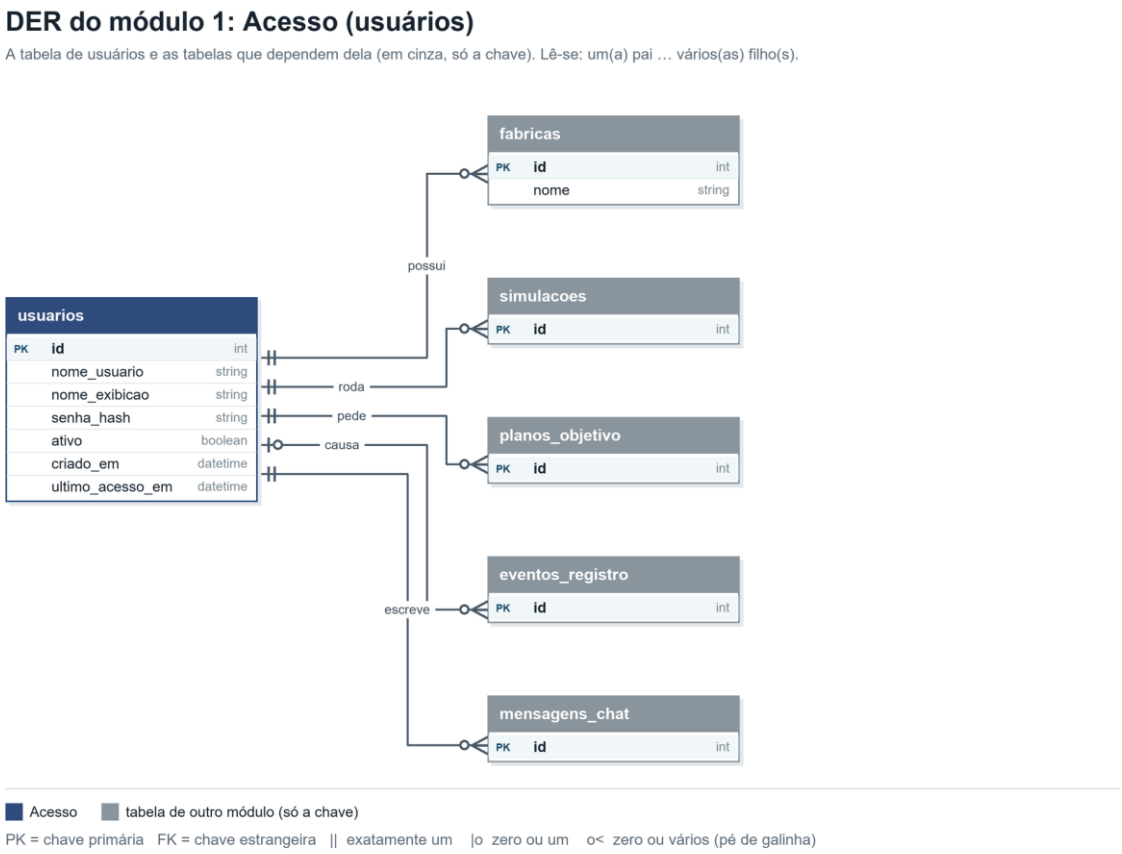


Figura 2 – DER do módulo Acesso (usuários).

5.3 Módulo Catálogo de máquinas

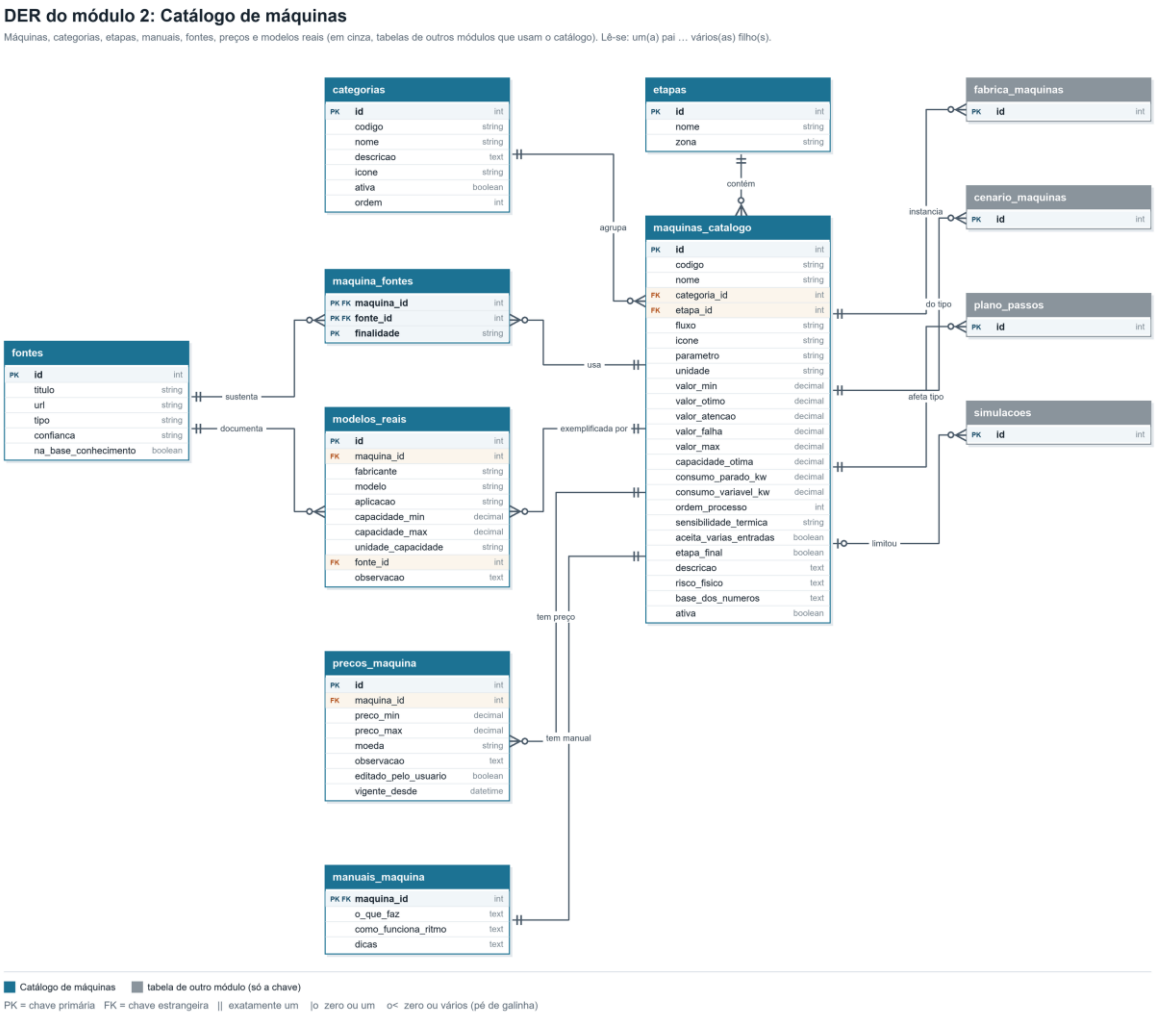


Figura 3 – DER do módulo Catálogo de máquinas (categorias, etapas, máquinas, manuais, fontes, preços e modelos reais).

5.4 Módulo Apoio ao usuário

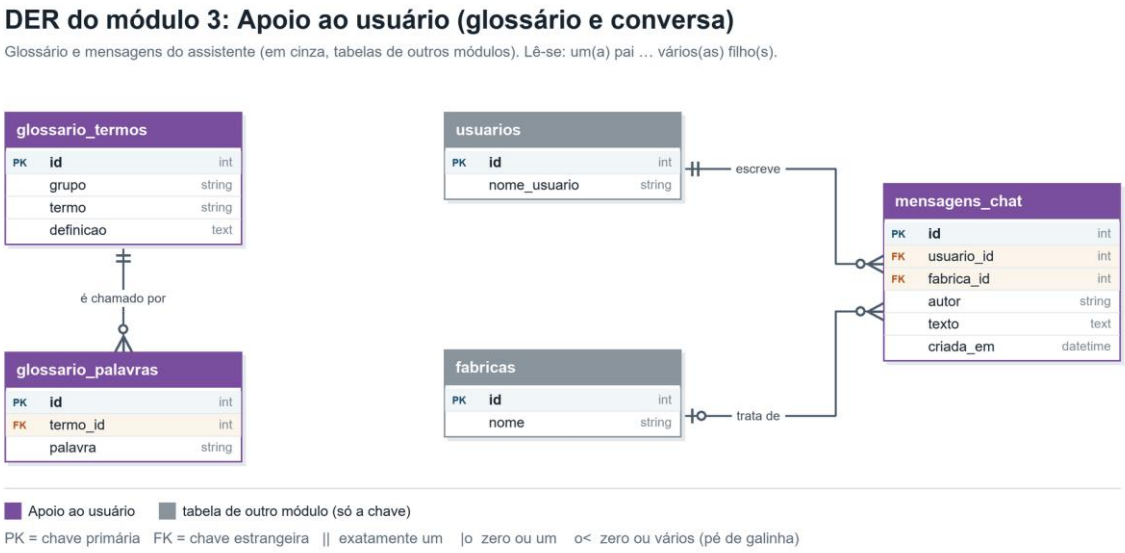


Figura 4 – DER do módulo Apoio ao usuário (glossário e conversa com o assistente).

5.5 Módulo Fábricas

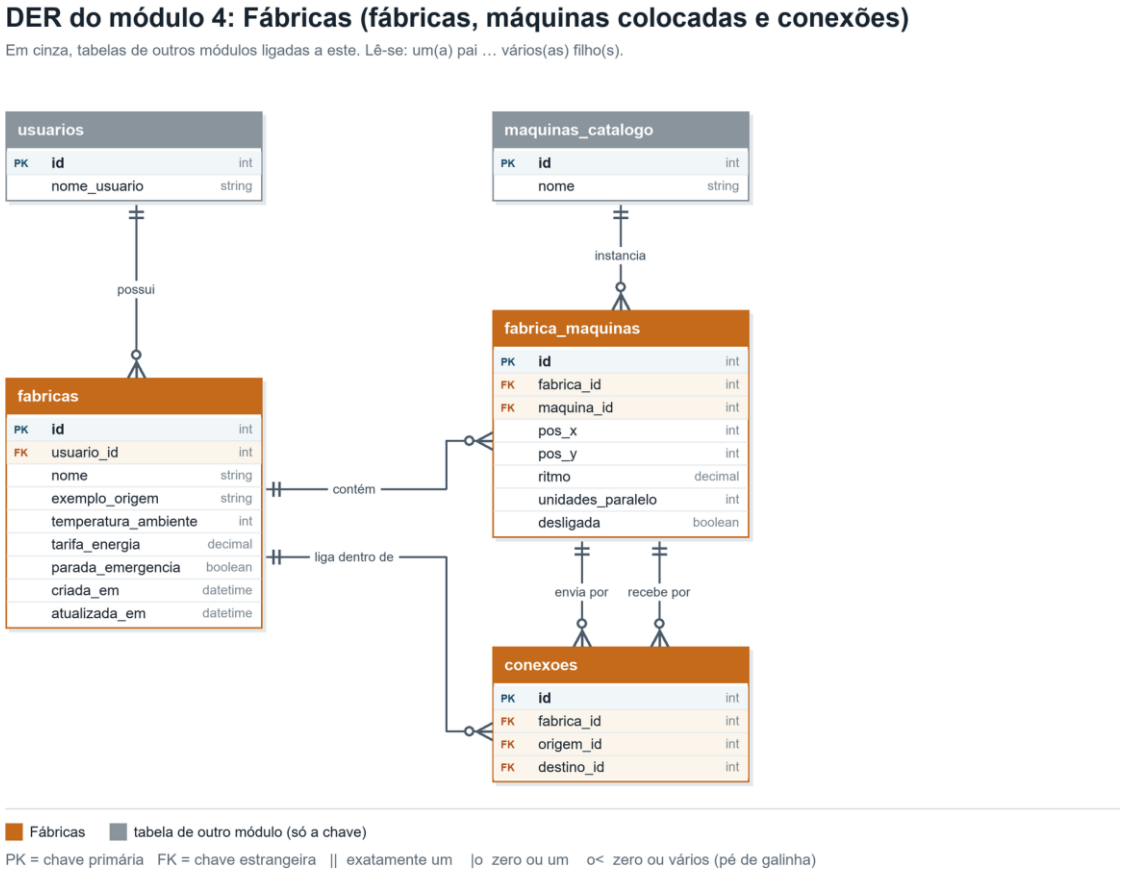


Figura 5 – DER do módulo Fábricas (fábricas, máquinas colocadas e conexões).

5.6 Módulo Simulação e resultados

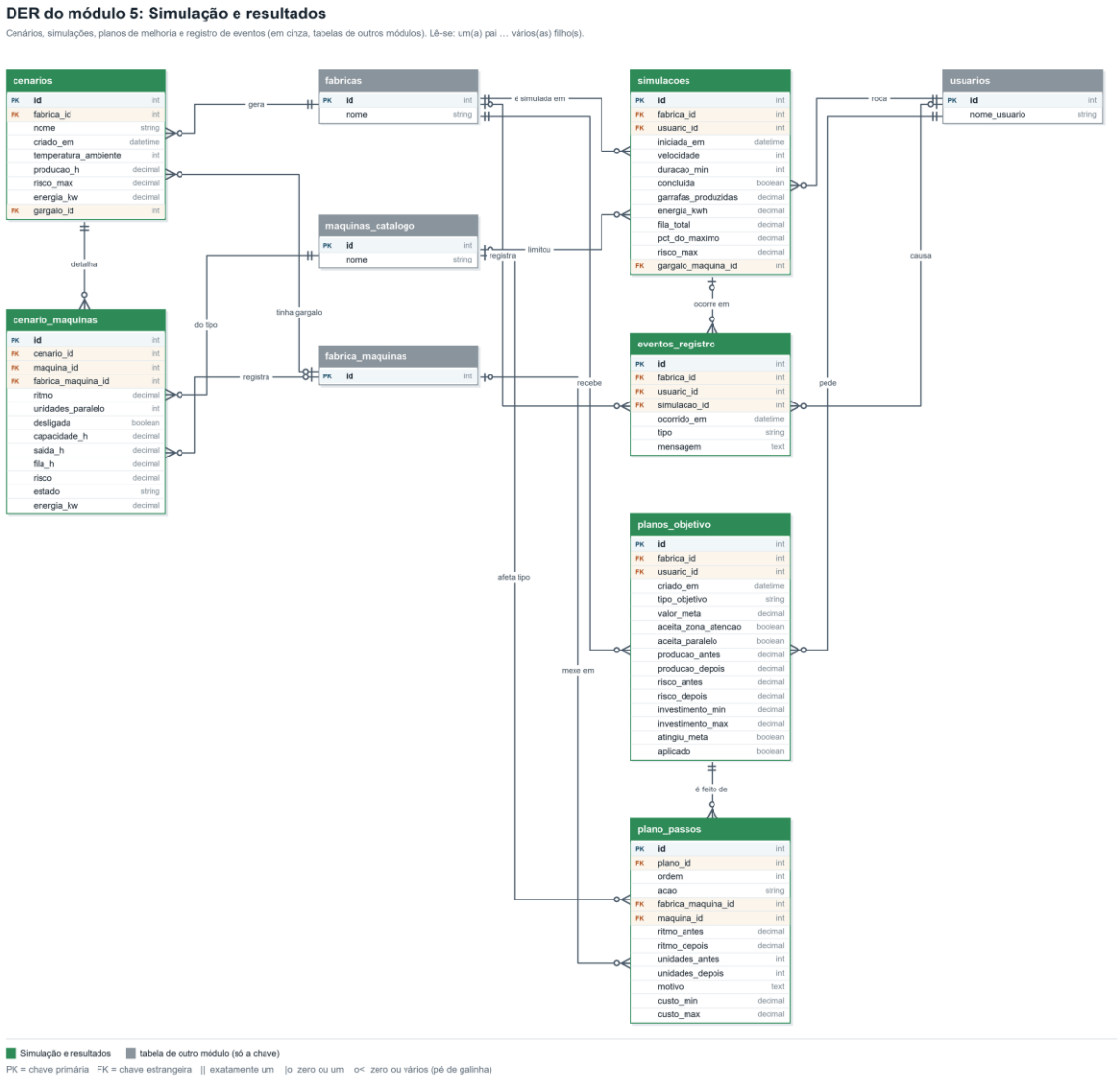


Figura 6 – DER do módulo Simulação e resultados (cenários, simulações, planos, passos e registro).

6. Normalização e decisões de projeto

O modelo está na **terceira forma normal (3FN)**:

- **1FN (valores atômicos):** nenhuma coluna guarda lista. As palavras que disparam um termo do glossário ficaram numa tabela própria (`glossario_palavras`), e as fontes de cada máquina em `maquina_fontes`, em vez de colunas com várias palavras ou links.
- **2FN (nada depende de parte da chave):** nas tabelas com chave composta (`maquina_fontes`) a única coluna extra faz parte da chave; as demais tabelas têm chave de uma coluna.
- **3FN (nada depende de outra coluna que não seja a chave):** a zona da planta baixa depende da etapa, então mora em `etapas` e não em cada máquina; a categoria mora em `categorias`; o nome do dono de uma fábrica é lido de `usuarios`, não repetido.

Uma escolha consciente: `cenario_maquinas` e `plano_passos` guardam **valores históricos** (o ritmo, a capacidade e o risco de quando a foto foi tirada). Eles parecem repetir dados da máquina, mas são a **foto** daquele momento: se a máquina mudar depois, a foto não pode mudar. Por isso não é redundância, é histórico. A primeira versão do backend guardava essa foto num campo de texto JSON; aqui virou uma tabela, que permite consultar e somar por SQL.

- **Identificadores:** chave primária numérica automática (`id`) em quase todas as tabelas; o texto "natural" (código da máquina, nome de usuário, URL da fonte) fica como valor **único**.
- **Chave composta para garantir a mesma fábrica:** `conexoes` aponta para `fabrica_maquinas` usando `(fabrica_id, id)`. Assim o banco recusa ligar uma máquina da fábrica A a uma da fábrica B, sem precisar de gatilho.
- **Regras de apagamento (ON DELETE):**
- `CASCADE` para o que não existe sem o pai (fábrica → máquinas, conexões, cenários, simulações, planos, eventos).
- `RESTRICT` para o catálogo: uma máquina usada em alguma fábrica não pode ser apagada; e um usuário com histórico de simulação ou de planos não pode ser apagado (o histórico é protegido).
- `SET NULL` onde o registro deve sobreviver sem o vínculo (a mensagem do chat continua se a fábrica for apagada; o gargalo de um cenário vira "desconhecido" se a máquina for removida).
- **Tipos:** valores monetários e de ritmo em `NUMERIC` (sem erro de arredondamento de ponto flutuante); datas em `TIMESTAMP`; verdadeiro/falso em `BOOLEAN`; textos curtos em `VARCHAR` com limite e textos longos em `TEXT`.
- **Regras no próprio banco (CHECK):** limites coerentes (mínimo ≤ ótimo ≤ atenção ≤ falha ≤ máximo), temperatura 0 a 45 °C, unidades em paralelo de 1 a 6, risco entre 0 e 1, estados e tipos só nos valores previstos. A tela também valida, mas o banco é a última barreira.
- **Senha:** só o hash (PBKDF2-SHA256 com sal) é guardado.
- **Fontes e confiança:** toda fonte tem `confianca` (`real`, `estimado` ou `hipotese`), seguindo a regra do projeto: dado real tem link; hipótese diz em que se baseia.

7. CRUD: Create, Read, Update e Delete

O CRUD foi demonstrado em **todas as 21 tabelas** pelo arquivo `sql/04_crud_exemplos.sql`, que cria dados de exemplo, lê, altera e apaga, deixando o banco como estava (a conferência final conta as linhas de cada tabela e compara com o estado inicial). O script `scripts/crud_demo.py` executa cada comando e mostra o resultado, e os testes automáticos repetem o ciclo em cada tabela.

Operação	Comando SQL	Exemplo no banco
Create	`INSERT`	cria uma fábrica, duas máquinas e a conexão entre elas
Read	`SELECT` (com `JOIN` e views)	lê a fábrica com suas máquinas e a ligação
Update	`UPDATE`	muda temperatura, tarifa, ritmo e unidades em paralelo
Delete	`DELETE`	apaga a fábrica e o que depende dela cai em cascata

Trecho real da execução de `python scripts/crud_demo.py`:

```
[READ] a fábrica com suas máquinas e a ligação (usa a view vw_fabricas_resumo e um JOIN)
  > SELECT * FROM vw_fabricas_resumo WHERE fabrica = 'Fábrica da Ana';
    fabrica_id | fabrica        | dono      | temperatura_ambiente | qtd_maquinas | qtd_conexoes |
qtd_simulacoes |...
    -----------+----------------+-----------+----------------------+--------------+--------------+----------
------+...
    4          | Fábrica da Ana | ana.souza | 28                   | 2            | 1            | 0
|...
  > SELECT mo.nome AS origem, md.nome AS destino FROM conexoes c JOIN fabrica_maquinas fo ON fo.id =
c.origem_id JO...
    origem    | destino
    ----------+-----------
    Enchedora | Rotuladora

[UPDATE] muda temperatura e tarifa da fábrica, o ritmo e as unidades em paralelo da rotuladora, desliga a
enchedora...
  > UPDATE fabricas SET temperatura_ambiente = 32, tarifa_energia = 0.95, atualizada_em = CURRENT_TIMESTAMP
WHERE n...
    1 linha(s) alterada(s)
  > UPDATE fabrica_maquinas SET ritmo = 225, unidades_paralelo = 2 WHERE fabrica_id = (SELECT id FROM
fabricas WHER...
    1 linha(s) alterada(s)
  > UPDATE fabrica_maquinas SET desligada = TRUE WHERE fabrica_id = (SELECT id FROM fabricas WHERE nome =
'Fábrica ...
    1 linha(s) alterada(s)
  > UPDATE usuarios SET nome_exibicao = 'Ana Souza Lima', ultimo_acesso_em = CURRENT_TIMESTAMP WHERE
nome_usuario =...
    1 linha(s) alterada(s)

[READ] conferindo
  > SELECT f.nome, f.temperatura_ambiente, f.tarifa_energia, m.codigo, fm.ritmo, fm.unidades_paralelo,
fm.desligada...
    nome           | temperatura_ambiente | tarifa_energia | codigo     | ritmo | unidades_paralelo |
desligada
    ---------------+----------------------+----------------+------------+-------+-------------------+-------
---
    Fábrica da Ana | 32                   | 0.95           | enchedora  | 200   | 1                 | 1
    Fábrica da Ana | 32                   | 0.95           | rotuladora | 225   | 2                 | 0
```

O banco também recusa dados inválidos, o que é parte do CRUD seguro: uma conexão entre máquinas de fábricas diferentes, uma temperatura fora de 0 a 45 °C, uma máquina do catálogo apagada enquanto está em uso ou um nome de usuário repetido geram erro de integridade (cobertos pelos testes da classe `Integridade`).

8. Dicionário de dados

Cada tabela, coluna, tipo, chave e regra de integridade. O mesmo conteúdo está em `docs/dicionario-de-dados.md`, gerado automaticamente do modelo (`scripts/modelo.py`).

Módulo: Acesso

usuarios

Pessoas que entram no sistema (login simples: usuário e senha, sem perfis)

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
`id`	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
`nome_usuario`	VARCHAR(50)	UQ	sim		Nome de login (único)
`nome_exibicao`	VARCHAR(100)		sim		Nome mostrado na tela
`senha_hash`	VARCHAR(255)		sim		Senha protegida (PBKDF2-SHA256 com sal); nunca a senha em texto
`ativo`	BOOLEAN		sim	verdadeiro	Falso = usuário desativado
`criado_em`	DATETIME		sim	agora	Quando a conta foi criada
`ultimo_acesso_em`	DATETIME		não		Último login (vazio se nunca entrou)

Módulo: Catálogo de máquinas

categorias

Tipos de máquina (grupos do catálogo): Insumos, Preparo, Térmico e gás, Envase, Pós-envase, Refrigeração

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
`id`	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
`codigo`	VARCHAR(30)	UQ	sim		Código curto (ex.: insumos, preparo, termico)
`nome`	VARCHAR(60)		sim		Nome mostrado na lista de máquinas e no manual
`descricao`	TEXT		sim		Explicação em palavras simples
`icone`	VARCHAR(30)		não		Nome do ícone usado na tela
`ativa`	BOOLEAN		sim	verdadeiro	Falso = a categoria some da lista e do manual
`ordem`	SMALLINT		sim	0	Ordem de exibição

etapas

Etapas do processo produtivo e a área (zona) da planta baixa em que cada uma aparece

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
nome	VARCHAR(60)	UQ	sim		Etapa do processo (ex.: Envase, Pós-envase)
zona	VARCHAR(40)		sim		Área da fábrica vista de cima onde a etapa fica

Regras de integridade:

- **Regra** ck_etapas_zona: `zona IN ('Recepção e preparo', 'Tratamento térmico e gás', 'Envase e expedição', 'Câmara fria')`

maquinas_catalogo

Catálogo: os tipos de máquina que podem ser colocados numa fábrica, com limites, capacidade e consumo

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
codigo	VARCHAR(40)	UQ	sim		Código único da máquina (ex.: enchedora, extratora)
nome	VARCHAR(80)		sim		Nome mostrado na tela
categoria_id	INTEGER	FK → categorias.id (RESTRICT)	sim		Categoria da máquina
etapa_id	INTEGER	FK → etapas.id (RESTRICT)	sim		Etapa do processo
fluxo	VARCHAR(10)		sim	geral	geral = bebidas em geral; suco = suco de laranja natural
icone	VARCHAR(30)		não		Nome do ícone usado na tela
parametro	VARCHAR(60)		sim		O que o ritmo controla (ex.: Cadência, Vazão, Rotação)
unidade	VARCHAR(15)		sim		Unidade do ritmo (ex.: gf/min, m³/h, RPM)
valor_min	NUMERIC(12, 3)		sim		Menor ritmo permitido
valor_otimo	NUMERIC(12, 3)		sim		Ritmo ótimo: trabalha bem, risco

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
					baixo
valor_atencao	NUMERIC(12, 3)		sim		Limite da zona de atenção (acima do ótimo)
valor_falha	NUMERIC(12, 3)		sim		A partir deste ritmo a máquina falha
valor_max	NUMERIC(12, 3)		sim		Maior ritmo permitido pelo controle
capacidade_otima	NUMERIC(12, 2)		sim		Garrafas por hora que uma unidade entrega no ritmo ótimo
consumo_parado_kw	NUMERIC(8, 3)		sim		Consumo com a máquina ligada e sem produzir (kW)
consumo_variavel_kw	NUMERIC(8, 3)		sim		Consumo adicional ao produzir no ótimo (kW)
ordem_processo	SMALLINT		não		Posição no processo (0 = começo, 9 = fim); vazio = pode ficar entre quaisquer etapas
sensibilidade_termica	VARCHAR(15)		sim	nenhuma	Como reage à temperatura ambiente
aceita_varias_entradas	BOOLEAN		sim	falso	Verdadeiro = recebe origens diferentes (ex.: misturador)
etapa_final	BOOLEAN		sim	falso	Verdadeiro = última etapa: nada vem depois
descricao	TEXT		sim		O que a máquina faz, em palavras simples
risco_fisico	TEXT		sim		O que pode dar errado fisicamente
base_dos_numeros	TEXT		sim		Nota de em que os números se baseiam (hipótese didática ou fonte)
ativa	BOOLEAN		sim	verdadeiro	Falso = máquina fora do catálogo

Regras de integridade:

- **Regra** `ck_maquinas_catalogo_limites`: `valor_min <= valor_otimo AND valor_otimo <= valor_atencao AND valor_atencao <= valor_falha AND valor_falha <= valor_max`
- **Regra** `ck_maquinas_catalogo_fluxo`: `fluxo IN ('geral', 'suco')`
- **Regra** `ck_maquinas_catalogo_capacidade`: `capacidade_otima > 0`
- **Regra** `ck_maquinas_catalogo_consumo`: `consumo_parado_kw >= 0 AND consumo_variavel_kw >= 0`
- **Regra** `ck_maquinas_catalogo_sensibilidade`: `sensibilidade_termica IN ('tanque', 'motor', 'frio', 'viscosidade', 'nenhuma')`

- **Regra** `ck_maquinas_catalogo_ordem`: `ordem_processo IS NULL OR (ordem_processo >= 0 AND ordem_processo <= 9)`

Índices: `ix_maquinas_catalogo_categoria_id`

manuais_maquina

Texto do Manual das máquinas (relação 1 para 1 com o catálogo)

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
`maquina_id`	INTEGER	PK, FK → maquinas_catalogo.id (CASCADE)	sim		Máquina a que o manual pertence (1 manual por máquina)
`o_que_faz`	TEXT		sim		Descrição completa da máquina
`como_funciona_ritmo`	TEXT		sim		Como o ritmo funciona
`dicas`	TEXT		sim		Dicas e cuidados

fontes

Fontes dos números (fichas, artigos, anúncios), cada uma com o nível de confiança

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
`id`	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
`titulo`	VARCHAR(300)		sim		Nome da fonte
`url`	VARCHAR(500)	UQ	não		Link da fonte (todo dado real tem link)
`tipo`	VARCHAR(40)		sim		Ex.: ficha técnica, artigo científico, anúncio de fornecedor
`confianca`	VARCHAR(10)		sim		real, estimado ou hipotese
`na_base_conhecimento`	BOOLEAN		sim	falso	Verdadeiro = documento consultado pelo assistente de conversa

Regras de integridade:

- **Regra** `ck_fontes_confianca`: `confianca IN ('real', 'estimado', 'hipotese')`

maquina_fontes

Liga máquinas e fontes (muitos para muitos)

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
`maquina_id`	INTEGER	PK, FK → maquinas_catalogo.id (CASCADE)	sim		Máquina
`fonte_id`	INTEGER	PK, FK → fontes.id (CASCADE)	sim		Fonte

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
finalidade	VARCHAR(10)	PK	sim		numeros = base dos limites e capacidade; preco = base do preço estimado

Regras de integridade:

- **Regra** ck_maquina_fontes_finalidade: finalidade IN ('numeros', 'preco')

Índices: ix_maquina_fontes_fonte_id

precos_maquina

Faixas de preço estimadas (nunca cotação), usadas para calcular o custo de uma melhoria

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
maquina_id	INTEGER	FK → maquinas_catalogo.id (CASCADE)	sim		Máquina
preco_min	NUMERIC(14, 2)		sim		Menor preço estimado de 1 unidade
preco_max	NUMERIC(14, 2)		sim		Maior preço estimado de 1 unidade
moeda	VARCHAR(3)		sim	BRL	Moeda (câmbio de referência R$ 5,50 por US$)
observacao	TEXT		não		Nota de como a faixa foi estimada
editado_pelo_usuario	BOOLEAN		sim	falso	Verdadeiro = ajustado nas Configurações
vigente_desde	DATETIME		sim	agora	Quando esta faixa passou a valer (permite guardar o histórico)

Regras de integridade:

- **Regra** ck_precos_maquina_faixa: preco_min >= 0 AND preco_min <= preco_max

Índices: ix_precos_maquina_maquina_id

modelos_reais

Modelos comerciais reais pesquisados (capacidades de fabricantes), com a fonte de cada um

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
maquina_id	INTEGER	FK → maquinas_catalogo.id	sim		Tipo de máquina do catálogo a que o

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
		(CASCADE)			modelo corresponde
fabricante	VARCHAR(80)		sim		Fabricante (ex.: Krones, Tetra Pak)
modelo	VARCHAR(80)		sim		Nome do modelo
aplicacao	VARCHAR(200)		não		Para que serve (ex.: água sem gás em PET)
capacidade_min	NUMERIC(12, 0)		não		Capacidade mínima, quando a fonte informa uma faixa
capacidade_max	NUMERIC(12, 0)		sim		Capacidade nominal máxima
unidade_capacidade	VARCHAR(20)		sim		garrafas/h, latas/h, recipientes/h ou L/h
fonte_id	INTEGER	FK → fontes.id (RESTRICT)	sim		De onde vem o dado (com link e confiança)
observacao	TEXT		não		Observações da pesquisa

Regras de integridade:

- **Único** `uq_modelos_reais_fabricante`: combinação de `fabricante, modelo`
- **Regra** `ck_modelos_reais_capacidade`: `capacidade_max > 0 AND (capacidade_min IS NULL OR capacidade_min <= capacidade_max)`

Índices: `ix_modelos_reais_maquina_id`

Módulo: Apoio ao usuário

glossario_termos

Glossário de palavras difíceis do sistema

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
grupo	VARCHAR(40)		sim		Produção, Risco e física, Energia e dinheiro, Processo de bebidas ou Sistema
termo	VARCHAR(80)	UQ	sim		A palavra explicada
definicao	TEXT		sim		Explicação em palavras simples

glossario_palavras

Palavras que disparam a explicação de um termo nas telas (um termo pode ter várias)

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
					automaticamente)
termo_id	INTEGER	FK → glossario_termos.id (CASCADE)	sim		Termo do glossário
palavra	VARCHAR(60)	UQ	sim		Palavra que aparece nas telas e ganha o sublinhado com a explicação

Índices: ix_glossario_palavras_termo_id

mensagens_chat

Conversa do usuário com o assistente

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
usuario_id	INTEGER	FK → usuarios.id (CASCADE)	sim		Dono da conversa
fabrica_id	INTEGER	FK → fabricas.id (SET NULL)	não		Fábrica sobre a qual se conversava
autor	VARCHAR(12)		sim		usuario ou assistente
texto	TEXT		sim		Conteúdo da mensagem
criada_em	DATETIME		sim	agora	Quando foi enviada

Regras de integridade:

- **Regra** ck_mensagens_chat_autor: autor IN ('usuario', 'assistente')

Índices: ix_mensagens_chat_usuario_criada

Módulo: Fábricas

fabricas

Fábricas salvas pelos usuários (cada uma é um mapa de máquinas ligadas entre si)

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
usuario_id	INTEGER	FK → usuarios.id (CASCADE)	sim		Dono da fábrica
nome	VARCHAR(100)		sim		Nome da fábrica
exemplo_origem	VARCHAR(20)		não		Exemplo de que partiu (bebidas ou suco); vazio = montada do zero
temperatura_ambiente	SMALLINT		sim	25	Temperatura do ambiente em °C (0 a 45)
tarifa_energia	NUMERIC(8, 4)		sim	0.85	Preço da energia em

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
					R$ por kWh
parada_emergencia	BOOLEAN		sim	falso	Verdadeiro = botão de emergência acionado
criada_em	DATETIME		sim	agora	Quando a fábrica foi criada
atualizada_em	DATETIME		sim	agora	Última alteração

Regras de integridade:

- **Regra** ck_fabricas_temperatura: temperatura_ambiente >= 0 AND temperatura_ambiente <= 45
- **Regra** ck_fabricas_tarifa: tarifa_energia >= 0
- **Único** uq_fabricas_usuario_nome: combinação de usuario_id, nome
- **Regra** ck_fabricas_exemplo: exemplo_origem IS NULL OR exemplo_origem IN ('bebidas', 'suco')

Índices: ix_fabricas_usuario_id

fabrica_maquinas

Máquinas colocadas numa fábrica (um tipo do catálogo + ritmo, posição e unidades em paralelo)

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
fabrica_id	INTEGER	FK → fabricas.id (CASCADE)	sim		Fábrica em que a máquina foi colocada
maquina_id	INTEGER	FK → maquinas_catalogo.id (RESTRICT)	sim		Tipo de máquina do catálogo
pos_x	INTEGER		sim	0	Posição horizontal no quadro (pixels)
pos_y	INTEGER		sim	0	Posição vertical no quadro (pixels)
ritmo	NUMERIC(12, 3)		sim		Ritmo atual (na unidade do parâmetro da máquina)
unidades_paralelo	SMALLINT		sim	1	Quantas máquinas iguais trabalham lado a lado (1 a 6)
desligada	BOOLEAN		sim	falso	Verdadeiro = máquina desligada

Regras de integridade:

- **Regra** ck_fabrica_maquinas_posicao: pos_x >= 0 AND pos_y >= 0
- **Único** uq_fabrica_maquinas_fabrica_id: combinação de fabrica_id, id
- **Regra** ck_fabrica_maquinas_paralelo: unidades_paralelo >= 1 AND unidades_paralelo <= 6
- **Regra** ck_fabrica_maquinas_ritmo: ritmo >= 0

Índices: ix_fabrica_maquinas_fabrica_id, ix_fabrica_maquinas_maquina_id

conexoes

Ligações entre duas máquinas da mesma fábrica (por onde o produto passa)

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
fabrica_id	INTEGER	FK → fabricas.id (CASCADE)	sim		Fábrica
origem_id	INTEGER	FK → fabrica_maquinas.id (CASCADE)	sim		Máquina de origem (a que envia o produto)
destino_id	INTEGER	FK → fabrica_maquinas.id (CASCADE)	sim		Máquina de destino (a que recebe o produto)

Regras de integridade:

- **Único** uq_conexoes_origem_destino: combinação de origem_id, destino_id
- **Chave estrangeira composta** fk_conexoes_origem: (fabrica_id, origem_id) → fabrica_maquinas (garante que as duas pontas pertencem à mesma fábrica)
- **Regra** ck_conexoes_sem_laco: origem_id <> destino_id
- **Chave estrangeira composta** fk_conexoes_destino: (fabrica_id, destino_id) → fabrica_maquinas (garante que as duas pontas pertencem à mesma fábrica)

Índices: ix_conexoes_destino_id, ix_conexoes_fabrica_id

Módulo: Simulação e resultados

cenarios

Fotos de uma fábrica simulada, para comparar antes e depois

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
fabrica_id	INTEGER	FK → fabricas.id (CASCADE)	sim		Fábrica fotografada
nome	VARCHAR(100)		sim		Nome do cenário (ex.: Situação inicial)
criado_em	DATETIME		sim	agora	Quando a foto foi tirada
temperatura_ambiente	SMALLINT		sim		Temperatura considerada (°C)
producao_h	NUMERIC(12, 2)		sim		Produção da fábrica (garrafas por hora)
risco_max	NUMERIC(5, 4)		sim		Risco da pior máquina (0 a 1)
energia_kw	NUMERIC(10, 3)		sim		Consumo total (kW)

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
gargalo_id	INTEGER	FK → fabrica_maquinas.id (SET NULL)	não		Máquina que era o gargalo

Regras de integridade:

- **Regra** ck_cenarios_valores: producao_h >= 0 AND energia_kw >= 0
- **Regra** ck_cenarios_risco: risco_max >= 0 AND risco_max <= 1

Índices: ix_cenarios_fabrica_id

cenario_maquinas

Situação de cada máquina dentro de um cenário (capacidade, saída, fila, risco e energia)

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
cenario_id	INTEGER	FK → cenarios.id (CASCADE)	sim		Cenário
maquina_id	INTEGER	FK → maquinas_catalogo.id (RESTRICT)	sim		Tipo da máquina
fabrica_maquina_id	INTEGER	FK → fabrica_maquinas.id (SET NULL)	não		Máquina na fábrica (vazio se ela foi removida depois)
ritmo	NUMERIC(12, 3)		sim		Ritmo na foto
unidades_paralelo	SMALLINT		sim		Unidades em paralelo na foto
desligada	BOOLEAN		sim	falso	Estava desligada?
capacidade_h	NUMERIC(12, 2)		sim		Quanto conseguia entregar (garrafas/h)
saida_h	NUMERIC(12, 2)		sim		O que realmente saía (garrafas/h)
fila_h	NUMERIC(12, 2)		sim	0	Excesso parado esperando esta máquina (garrafas/h)
risco	NUMERIC(5, 4)		sim		Risco de falha (0 a 1)
estado	VARCHAR(10)		sim		normal, alerta, critico, falha ou desligada
energia_kw	NUMERIC(10, 3)		sim		Consumo (kW)

Regras de integridade:

- **Regra** ck_cenario_maquinas_risco: risco >= 0 AND risco <= 1
- **Regra** ck_cenario_maquinas_estado: estado IN ('normal', 'alerta', 'critico', 'falha', 'desligada')

Índices: ix_cenario_maquinas_cenario_id

simulacoes

Resultado final de cada vez que o usuário roda o Play

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
fabrica_id	INTEGER	FK → fabricas.id (CASCADE)	sim		Fábrica simulada
usuario_id	INTEGER	FK → usuarios.id (RESTRICT)	sim		Quem rodou o Play
iniciada_em	DATETIME		sim	agora	Quando o Play foi iniciado
velocidade	SMALLINT		sim		Velocidade usada: 1, 5, 20 ou 60 vezes
duracao_min	INTEGER		sim		Duração simulada em minutos (30, 60 ou 480)
concluida	BOOLEAN		sim	verdadeiro	Falso = parada antes do fim
garrafas_produzidas	NUMERIC(14, 0)		sim		Total produzido no período
energia_kwh	NUMERIC(12, 2)		sim		Energia gasta no período (kWh)
fila_total	NUMERIC(14, 0)		sim	0	Garrafas paradas na fila ao final
pct_do_maximo	NUMERIC(6, 2)		sim		Percentual do máximo possível (tudo no ótimo)
risco_max	NUMERIC(5, 4)		sim		Maior risco visto (0 a 1)
gargalo_maquina_id	INTEGER	FK → maquinas_catalogo.id (SET NULL)	não		Tipo de máquina que mais limitou a fábrica

Regras de integridade:

- **Regra** `ck_simulacoes_duracao`: `duracao_min > 0`
- **Regra** `ck_simulacoes_risco`: `risco_max >= 0 AND risco_max <= 1`
- **Regra** `ck_simulacoes_velocidade`: `velocidade IN (1, 5, 20, 60)`
- **Regra** `ck_simulacoes_valores`: `garrafas_produzidas >= 0 AND energia_kwh >= 0 AND fila_total >= 0`

Índices: `ix_simulacoes_fabrica_id`

planos_objetivo

Planos do tipo 'quero +10% de produção': o que fazer, o resultado esperado e o investimento

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
fabrica_id	INTEGER	FK → fabricas.id	sim		Fábrica

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
		(CASCADE)			
usuario_id	INTEGER	FK → usuarios.id (RESTRICT)	sim		Quem pediu o plano
criado_em	DATETIME		sim	agora	Quando o plano foi calculado
tipo_objetivo	VARCHAR(5)		sim		pct = aumentar em %; abs = chegar a uma produção; risk = reduzir o risco
valor_meta	NUMERIC(12, 2)		não		Valor pedido (% ou garrafas/h); vazio para risk
aceita_zona_atencao	BOOLEAN		sim	falso	Permitiu operar acima do ótimo?
aceita_paralelo	BOOLEAN		sim	falso	Permitiu usar mais máquinas em paralelo?
producao_antes	NUMERIC(12, 2)		sim		Produção antes do plano (garrafas/h)
producao_depois	NUMERIC(12, 2)		sim		Produção prevista depois do plano
risco_antes	NUMERIC(5, 4)		sim		Risco máximo antes (0 a 1)
risco_depois	NUMERIC(5, 4)		sim		Risco máximo depois (0 a 1)
investimento_min	NUMERIC(14, 2)		sim	0	Investimento estimado mínimo (R$)
investimento_max	NUMERIC(14, 2)		sim	0	Investimento estimado máximo (R$)
atingiu_meta	BOOLEAN		sim		O plano consegue chegar à meta?
aplicado	BOOLEAN		sim	falso	O usuário aplicou o plano na fábrica?

Regras de integridade:

- **Regra** `ck_planos_objetivo_risco`: `risco_antes >= 0 AND risco_antes <= 1 AND risco_depois >= 0 AND risco_depois <= 1`
- **Regra** `ck_planos_objetivo_tipo`: `tipo_objetivo IN ('pct', 'abs', 'risk')`
- **Regra** `ck_planos_objetivo_investimento`: `investimento_min >= 0 AND investimento_min <= investimento_max`

Índices: `ix_planos_objetivo_fabrica_id`

plano_passos

Passos de um plano (qual máquina mexer, de quanto para quanto, por quê e quanto custa)

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
plano_id	INTEGER	FK → planos_objetivo.id (CASCADE)	sim		Plano
ordem	SMALLINT		sim		Posição do passo no

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
					plano (1, 2, 3...)
acao	VARCHAR(10)		sim		ligar, paralelo ou ritmo
fabrica_maquina_id	INTEGER	FK → fabrica_maquinas.id (SET NULL)	não		Máquina da fábrica afetada (vazio se foi removida)
maquina_id	INTEGER	FK → maquinas_catalogo.id (RESTRICT)	sim		Tipo da máquina afetada
ritmo_antes	NUMERIC(12, 3)		não		Ritmo antes (ação ritmo)
ritmo_depois	NUMERIC(12, 3)		não		Ritmo depois (ação ritmo)
unidades_antes	SMALLINT		não		Unidades em paralelo antes (ação paralelo)
unidades_depois	SMALLINT		não		Unidades em paralelo depois (ação paralelo)
motivo	TEXT		sim		O porquê do passo, em palavras simples
custo_min	NUMERIC(14, 2)		não		Custo estimado mínimo do passo (R$)
custo_max	NUMERIC(14, 2)		não		Custo estimado máximo do passo (R$)

Regras de integridade:

- **Regra** `ck_plano_passos_ordem`: `ordem >= 1`
- **Único** `uq_plano_passos_plano_ordem`: combinação de `plano_id, ordem`
- **Regra** `ck_plano_passos_acao`: `acao IN ('ligar', 'paralelo', 'ritmo')`

Índices: `ix_plano_passos_plano_id`

eventos_registro

Registro (log) do que o usuário faz e do que acontece durante a simulação

Coluna	Tipo	Chave	Obrigatório	Padrão	Descrição
id	INTEGER	PK	sim		Identificador da linha (gerado automaticamente)
fabrica_id	INTEGER	FK → fabricas.id (CASCADE)	não		Fábrica a que o evento se refere (vazio = evento geral)
usuario_id	INTEGER	FK → usuarios.id (SET NULL)	não		Quem causou o evento
simulacao_id	INTEGER	FK → simulacoes.id (CASCADE)	não		Simulação em que ocorreu (se foi durante o Play)
ocorrido_em	DATETIME		sim	agora	Quando aconteceu
tipo	VARCHAR(5)		sim	info	info, ok, warn ou bad (cor da linha no Registro)
mensagem	TEXT		sim		O que aconteceu

Regras de integridade:

- **Regra** `ck_eventos_registro_tipo`: `tipo IN ('info', 'ok', 'warn', 'bad')`

Índices: `ix_eventos_registro_fabrica_ocorrido`

9. Como executar

- **Ver o banco:** abrir `digital_twin.db` no DB Browser for SQLite (gratuito, https://sqlitebrowser.org).
- **Ver o CRUD:** `python scripts/crud_demo.py` (só precisa do Python).
- **Rodar os testes:** `python -m unittest discover -s testes -v`.
- **Criar do zero em SQLite:** `sqlite3 meu_banco.db < sql/01_schema_sqlite.sql`, depois `02_dados_iniciais.sql` e `03_views.sql`.
- **Criar em PostgreSQL com Docker:** `docker compose up -d` (sobe o servidor e já cria tudo) e `python scripts/testar_postgres.py` (confere as tabelas, roda o CRUD completo e os testes de integridade).
- **Criar num PostgreSQL já instalado:** `createdb digital_twin` e `psql digital_twin -f sql/01_schema_postgres.sql`, depois `02_dados_iniciais.sql`, `02b_ajustar_sequencias_postgres.sql` e `03_views.sql`.
- **Regerar os arquivos a partir do modelo:** `python scripts/gerar_banco.py` (SQL, banco e dicionário) e `python scripts/gerar_der.py` (diagramas).